Vision Robotique - TP2 - OpenCV

L'objectif de ce TP est la mise en oeuvre de la bibliothèque OpenCV pour le traitement des images et de la vidéo. Nous allons aussi mettre en oeuvre des outils de profiling qui nous permettront d'évaluer la qualité de notre code ainsi que ses performances.

CMake & Make

Comme nous l'avons vu lors du premier TP, nous allons, pour construire l'exécutable de nos programmes et des suivants, utiliser un système de construction logiciel : **CMake**.

En premier lieu, nous créons notre répertoire projet

> mkdir tp2-vision

et un sous-répertoire où on rangera nos codes sources :

> cd tp2-vision
> tp2-vision$mkdir src

Hello world

On commence avec le plus simple des codes source, le fameux "hello world", à entrer dans un fichier *hello.cpp* placé dans le dossier *src* en utilisant l'éditeur texte de votre choix (nano, vi, emacs, gedit ...). Une fois que notre espace de travail sera configuré à l'étape suivante, on pourra bien sûr utiliser notre EDI préféré.

```
#include <stdio.h>
#include <opencv2/opencv.hpp>
using namespace cv;
int main(int argc, char** argv )
{
    if ( argc != 2 )
    {
        printf("usage: DisplayImage.out <Image_Path>\n");
        return -1;
    }
    Mat image;
    image = imread( argv[1], 1 );
/*    if ( !image.data )
    {
        printf("No image data \n");
        return -1;
    }
*/
    namedWindow("Display Image", WINDOW_AUTOSIZE );
    imshow("Display Image", image);
    waitKey(0);
    return 0;
}
```

Dans le répertoire projet, on peut alors créer notre fichier CMakeLists.txt :

```
project(ocvproj)

set(EXECUTABLE_OUTPUT_PATH bin/${CMAKE_BUILD_TYPE})

find_package(OpenCV REQUIRED)

include_directories(${OpenCV_INCLUDE_DIRS})

add_executable(hello src/hello.cpp)

target_link_libraries(hello ${OpenCV_LIBS})
```

À partir du fichier descriptif du projet (CMakeLists.txt) et des informations sur la configuration logicielle de la machine effectuant la compilation (listées dans le fichier CMakeCache.txt, lequel sera prérempli lors de la phase d'analyse de la configuration), CMake peut générer les fichiers suivants :

- un fichier Makefile
- un fichier de projet d'EDI
- ou une combinaison des deux pour les EDI supportant les fichiers Makefile; c'est la cas de l'EDI CodeBlocks que nous allons utiliser ici.

Pour obtenir la liste des générateurs de projet d'EDI :

> cmake –help

Dans notre situation :

> cmake . -G'CodeBlocks - Unix Makefiles'

Il est dès à présent possible de lancer un build de notre code à l'aide de la commande

> make

dans le répertoire du projet, ou en lancant le build de notre projet à partir de notre EDI CodeBlocks. **Avant cela**, nous allons indiquer que nous souhaitons que la construction se fasse en mode *Release* et positionnons la variable *CMAKE_BUILD_TYPE* dans le fichier CMakeCache.txt :

```
CMAKE_BUILD_TYPE:STRING=Release
```

Nous pouvons maintenant exécuter le programme.

> tp2-vision/bin/Release$./hello lena.jpg

A noter que notre code prend un fichier image en argument, ici *lena.jpg*.

Optionnel : Pour voir apparaître CMakeCache.txt dans le répertoire virtuel CMake Files de CodeBlocks, rajouter avant la fin du fichier nom_de_projet.cbp :

```
<Unit filename="/home/senn/work_C/td1-mod/CMakeCache.txt">
            <Option virtualFolder="CMake Files\"/>
        </Unit>
```

Profiling

Pour procéder au profiling, nous allons dans le code commenter la ligne *//waitKey(0);*.
Une fois l'exécutable généré, nous procédons à son profiling à l'aide des outils **Valgrind / Cachegrind** dont on rappelle l'utilisation ici :

```
valgrind --tool=cachegrind --time-stamp=yes ./hello ./lena.jpg
kcachegrind cachegrind.out.xxxxx &
valgrind --tool=massif --time-unit=B ./hello ./lena.jpg
ms_print massif.outxxxxxx
```

A noter que les *time-stamps* ajoutés par valgrind sont donnés avec les unités suivantes :

```
 --time-stamp=<yes|no> [default: no]

When enabled, each message is preceded with an indication of the elapsed wallclock time since sta
rtup, expressed as days, hours, minutes, seconds and milliseconds.
```

Si l'on s'intéresse au graphe d'appel des fonctions, on appellera plutôt l'outil **callgrind** (il faudra alors compiler en mode **Debug** pour que le lien avec le code source soit établi) :

```
valgrind --tool=callgrind --time-stamp=yes ./hello ./lena.jpg
kcachegrind callgrind.out.xxxxx &
```

Plus d'information sur le site dédié : https://www.valgrind.org/

La bibliothèque OpenCV se décline aussi en python, et dans ce langage, le code équivalent au précédent s'écrit comme suit :

```
import numpy as np
import cv2
import sys

if len(sys.argv) != 2:
    print 'usage: DisplayImage.out <Image_Path>'
    exit()

# Load a color image
img = cv2.imread(sys.argv[1])
# Display image in a window
cv2.imshow('image',img)
#cv2.waitKey(0)
cv2.destroyAllWindows()
```

Le profiling du code python s'opère comme pour l'exécutable obtenu à partir du code *C/C++* :

```
valgrind --tool=cachegrind --time-stamp=yes python hello.py lena.jpg
```

Relever et commenter les résultats : quel est le meilleur code ? pourquoi ?
(On s'intéresse en première approche au temps d'exécution des exécutables - appelés aussi *binaires* - , au nombre d'accès à la mémoire, pour les instructions et les données, et à l'empreinte mémoire.)
Pourquoi a-t-on commenté les lignes *"...waitkey(0)"* dans les deux versions du code ?

CImg

On souhaite maintenant comparer les performances de notre exécutable avec celles de l'exécutable équivalent que nous avons généré en utilisant la bibliothèque **CImg** à l'occasion du premier TP, et dont on rappelle le code ici :

```
#include <iostream>
#include "libcimg/CImg.h"
using namespace cimg_library;

// Main procedure
int main(int argc,char **argv) {
  // Read image filename from the command line (or set it to "lena.jpg" if option '-i' is not pro
vided)
  const char* file_i = cimg_option("-i","lena.jpg","Input image");

  // Load an image
  const CImg<unsigned char> image = CImg<>(file_i);

  CImgDisplay disp(image,"lena");
  image.display(disp);
//  image.display("lena");
  return 0;
}
```

Le fichier *CMakeLists.txt* à utiliser :

```
#Déclaration du projet
project(cimgproj)
set(EXECUTABLE_OUTPUT_PATH bin/${CMAKE_BUILD_TYPE})
include_directories(include)
#Déclaration de l'exécutable
add_executable(cimgtest src/cimg_hello.cpp)
#Configuration de l'édition de liens
target_link_libraries(
        cimgtest
        pthread
        X11
)
```

Lancer un profiling à l'aide de valgrind / cachegrind sur cet exécutable.
Relever et commenter les résultats obtenus.
Quelles comparaisons faire entre les bibliothèques **CImg** et **OpenCV** ?

Capture et traitements vidéo

Nous allons écrire un programme qui permet d'ouvrir un flux vidéo en provenance de la caméra de l'ordinateur, d'en transformer les images en niveaux de gris, puis de les binariser, et d'afficher les résultats de ces traitements. Pour construire l'exécutable avec le commande *make* on pensera à modifier comme il se doit le fichier *CMakeLists.txt* du projet :

```
set(EXECUTABLE_OUTPUT_PATH bin/${CMAKE_BUILD_TYPE})

find_package(OpenCV REQUIRED)
```

```
include_directories(${OpenCV_INCLUDE_DIRS})

add_executable(hello src/hello.cpp)
add_executable(capt src/capture.cpp)

target_link_libraries(hello ${OpenCV_LIBS})
target_link_libraries(capt ${OpenCV_LIBS})
```

Code source *capture.cpp* :

```
#include <stdio.h>
#include <iostream>
#include <opencv2/opencv.hpp>
using namespace std;
using namespace cv;

int main(int argc, char** argv )
{
VideoCapture cap(0);//pour capturer depuis la cam

if ( !cap.isOpened())
{
cout << "Error retrieving frame \n";
    cout << "Could not open reference " << endl;
return -1;
}

while(true) //Boucle principale
{
Mat imgOriginal, imgGRAY, imgBIN;

cap >> imgOriginal;//on capture une frame depuis le flux video

if( imgOriginal.empty() )
{
cout << " < < < Frame Empty ! > > > ";
break;
}

cvtColor(imgOriginal, imgGRAY, cv::COLOR_RGB2GRAY);//Convert the image to gray levels
threshold( imgGRAY, imgBIN, 128, 255, 0 );//threshold the gray image as to binarize it

//show the images
imshow("couleurs", imgOriginal); //show the original image
imshow("niveaux de gris",imgGRAY);
imshow("binaire", imgBIN);

if (waitKey(30) == 27) //wait for 'esc' key press for 30ms. If 'esc' key is pressed, break loop
       {
            cout << "ESC key pressed by user - video thread terminates" << endl;
            break;
       }
}//fin du while boucle principale

    return 0;
}
```

Pour information, l'équivalent en python du code précédent est donné ci-dessous. On préférera travailler en C/C++.

```
import numpy as np
import cv2

cap = cv2.VideoCapture(0)

while(True):
    # Capture frame-by-frame
    ret, frame = cap.read()

    # Our operations on the frame come here
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # global thresholding
    ret1,th1 = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)

    # Display the resulting frame
    cv2.imshow('couleurs',frame)
    cv2.imshow('niveaux de gris',gray)
    cv2.imshow('binaire',th1)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# When everything done, release the capture
cap.release()
cv2.destroyAllWindows()
```

Traiter un flux vidéo en provenance d'un fichier se fera assez simplement en remplaçant dans le code *C++* la ligne *VideoCapture cap(0);* comme suit :

```
VideoCapture cap("Meet_Crowd2.avi");
```

Et si l'on souhaite passer le nom du fichier vidéo en ligne de commande, on remplacera le début du *main.cpp* par :

```
    if (argc != 2)
    {
        cout << "Not enough parameters" << endl;
        return -1;
    }
    const string source = argv[1];//the source video file name
    VideoCapture cap(source);     //open the input file
```

Afin d'évaluer les performances de notre programme on se propose de suivre deux approches :
1- mesurer le temps nécessaire au traitement d'une frame.
2- mesurer la cadence image à laquelle le programme permet d'arriver.

Pour mesurer le temps d'exécution il est possible d'utiliser la bibliothèque *chrono* (préférée à l'antédiluvienne bibliothèque *ctime*), ou, lorsque l'on utilise la bibliothèque *OpenCV*, les fonctions de la classe *cv::TicketMeter*, comme illustré sur les deux exemples à suivre.

```
...
```

```
include <chrono>
include <iomanip>
...

//Record start time
auto start = std::chrono::high_resolution_clock::now();

.../...

// Record end time
auto finish = std::chrono::high_resolution_clock::now();

std::chrono::duration<double> elapsed = finish - start;
std::cout << "Elapsed time: " << elapsed.count() << setprecision(10) << " s\n";
```

```
TickMeter tm;

tm.start();

.../...

tm.stop();

cout << "Total time: " << tm.getTimeSec() << endl;

tm.reset();
```

Si vous avez un message d'erreur à la compilation avec la librairie *chrono*, il est à noter que la lib chrono a été écrite en C++ version 2011 du standard, donc cela nous contraint à compiler notre code avec une version du compilateur qui supporte ce standard et dont on doit l'informer en ajoutant la commande suivante au **CMakeLists.txt** :

> add_compile_options(-std=c++11)

LIVRABLE 1 :
Le programme doit être capable
de lire le flux vidéo en provenance d'une caméra reliée à l'ordinateur.
de convertir l'image en niveaux de gris.
de seuiller cette image pour obtenir une image binaire.
d'afficher le flux vidéo en niveau de gris et binarisé sur des fenêtres séparées.
de mesurer et d'afficher le temps nécessaire au traitement d'une frame.

Détection de présence

Niveau 1

On se propose d'imaginer les traitements à réaliser pour suivre le déplacement d'une personne devant l'objectif de la caméra. Une approche simple consiste à comparer l'image saisie à une image représentant le fond de la scène. On vous demande ici de mettre en œuvre ces traitements et d'évaluer leur efficacité, ainsi que leurs performances. Indiquez précisémment quelles sont les conditions d'expérience dans lesquelles vous vous placez et leur niveau de difficulté.

LIVRABLE 2 :
Le programme doit être capable de détecter la présence d'une personne devant l'objectif.
Le programme doit incruster sur le flux video original une boîte encadrant la personne détectée, et les coordonnées du centre de cette boîte.

Tester vos traitements sur les vidéos à disposition en provenance de caméras de surveillance.
Imaginer une méthode pour comparer qualitativement et quantitativement les différentes versions de votre code

On donne ci-dessous des éléments de langage pour réaliser les fonctions demandées.

```
Moments oMoments = moments(imgThresholded);//Calcul des moments d'une image binaire
double dM01 = oMoments.m01;//retourne la somme des y de la forme (ou la valeur du pixel vaut 1 da
ns l'image binaire)
double dM10 = oMoments.m10;//retourne la somme des x de la forme
double dArea = oMoments.m00;//retourne la nombre de pixels dans la forme

cv::Rect boundRect = boundingRect(imgThresholded);//Calcul de la boîte englobante d'une image bin
aire
rectangle(image,boundRect, cv::Scalar(0, 255, 0), 2);//incrustation de la boîte dans l'image de d
épart

//pour incruster une ligne contenant du texte et des valeurs de variables
std::ostringstream oss (std::ostringstream::ate);
std::string result;
//we create the following variable to convert the variables values into string
oss << "variable 1 =" << variable1 << "variable 2 =" << variable2;
result = oss.str();
//we write the string on the output image
putText(image, result, cvPoint(10,20),FONT_HERSHEY_SIMPLEX,0.5,CV_RGB(255,0,0),2,8,false);
```

Niveau 2

Plusieurs personnes peuvent maintenant se présenter devant l'objectif.

LIVRABLE 3 :
Le programme doit détecter la présence de plusieurs personnes devant l'objectif.
Le programme incruste sur le flux vidéo original des boîtes encadrant les personnes répérées, ainsi que le nombre de personnes.